

شرح تقني مُبسَّط — للمبتدئين

Crowd XI

كيف بنينا تطبيقاً لا ينهار حين يدخله ألف شخص دفعة واحدة

رابط واحد يشاركه المُقدِّم، كل مشاهد يبني فريق فانتازي حقيقي، وتظهر على الشاشة مباشرةً التشكيلة التي اختارها الجمهور. هذا المستند يشرح المشروع من الصفر، ويركّز على الجزء الأهم: الجِئِل التي تجعله يصمد تحت ضغط الزحام.

لمن هذا المستند؟ لشخص لا يعرف البرمجة كثيراً. كل مصطلح تقني سيُشرح بمثال من الحياة اليومية قبل أن نرى الكود.

المحتويات

١ — ما هو المشروع؟ القصة كاملة في صفحة

الفكرة والأدوار الثلاثة

٢ — كيف تسير المعلومة داخل النظام

مخطط بسيط

٣ — الكارثة المُحتملة: حساب الأرقام

لماذا ينهار التطبيق أصلاً

٤ — الحيلة الأولى: تجميع الطلبات (Debouncing)

أهم فصل في المستند

٥ — الحيلة الثانية: منع الطلب المكرر

قفل «قيد التنفيذ»

٦ — الحيلة الثالثة: شبكة الأمان الصامتة

Polling ذي

٧ — الحيلة الرابعة: ذاكرة مؤقتة عند الحافة

ألف طلب → قراءة واحدة

٨ — الحيلة الخامسة: ترويض واجهة FPL

من ٢.٣ ميغا إلى ١٠٠ كيلو

٩ — الحيلة السادسة: كابح المُسيئين

Rate limiting

١٠ — الحيلة السابعة: واجهة خفيفة في المتصفح

useMemo و LIST_CAP

١١ — الأمان: لا تثق بالمتصفح أبداً

الكوكي المُوقَّع

١٢ — الحيلة النهائية

جدول الأرقام والخلاصة

قاعدة تتكرر في كل فصل

كل تحسين في هذا المستند يجيب على سؤال واحد: «ما الذي يتضاعف عندما يزيد عدد المشاهدين؟» إن وجدت شيئاً يتضاعف مرتين في آنٍ واحد (عدد المشاهدين $\times$ عدد الفرق مثلاً)، فهذا هو المكان الذي سينكسر.

ما هو المشروع؟

تخيّل مُقدِّماً على «تويتش» أو «يوتيوب» قبل انطلاق جولة الدوري الإنجليزي. يقول لجمهوره: «ادخلوا هذا الرابط واختاروا فريقكم». خلال دقائق، تظهر على شاشته تشكيلة واحدة مكوّنة من أكثر ١١ لاعباً اختارهم الجمهور، ومعها الكابتن الذي صوّت له الأغلبية. هذا هو Crowd XI.

الأدوار الثلاثة

الدور	ماذا يفعل	الصفحة في التطبيق
المُقدِّم (Host)	ينشئ «حوض» (Pool) ويحصل على رابط ومُعزّف قصير يقرؤه بصوته على الهواء.	/ → /p/[id]/live
المشاهد (Viewer)	يفتح الرابط، يختار ١٥ لاعباً ضمن ميزانية حقيقية، ثم يُرسل.	/p/[id]
الشاشة (Board)	تعرض التشكيلة الجماعية وتحدّث لحظياً كلما وصل فريق جديد.	/p/[id]/live

ثلاث حقائق تُفسّر كل قرار هندسي لاحق

١) لا يوجد تسجيل دخول. هوية المشاهد مجرّد رقم عشوائي محفوظ في متصفحه. هذا يجعل المشاركة سهلة (نقرة واحدة)، لكنه يعني أن الخادم لا يستطيع الوثوق بأي شيء يرسله المتصفح — وسنرى في الفصل ١١ كيف عالجنّا ذلك.

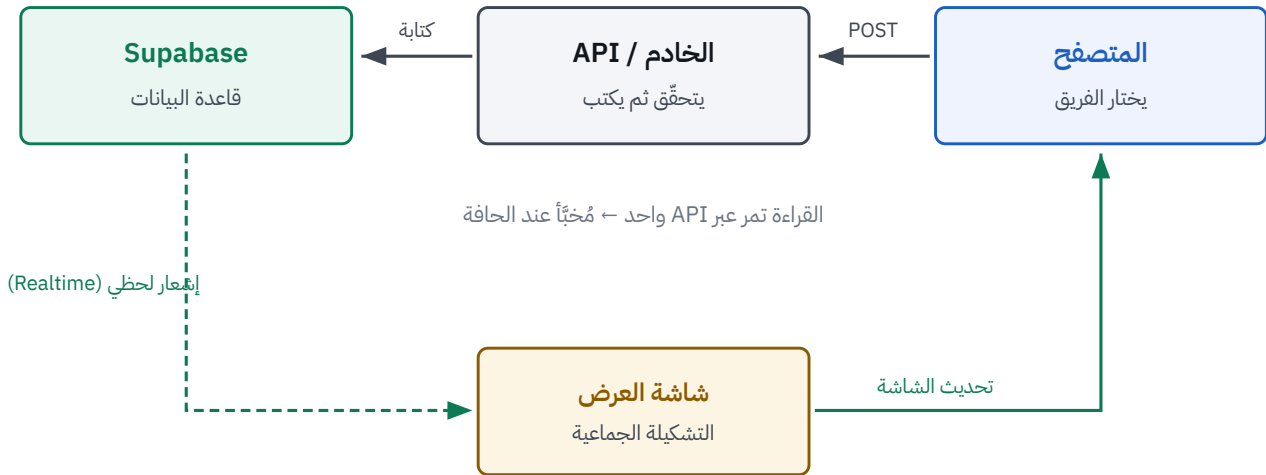
٢) البيانات تأتي من واجهة FPL الرسمية. أسعار اللاعبين، النقاط، والمواعيد النهائية كلها من خادم خارجي مجاني ومفتوح... ويمكن أن يتعطل في أي لحظة. لذلك كل مسار يلმسه يجب أن يتدهور بلطف لا أن ينهار.

٣) اللحظة الحرجة معروفة سلفاً. الجمهور لا يأتي بالتساوي طوال اليوم؛ بل يأتي كلّ في الدقائق الأخيرة قبل الموعد النهائي للجولة. التطبيق مُصمّم لتلك الدقائق تحديداً، لا للمعدّل اليومي.

كيف تسير المعلومة داخل النظام

٢

ثلاثة أطراف فقط: المتصفح، الخادم (Next.js)، وقاعدة البيانات (Supabase). القاعدة الذهبية: المتصفح لا يكتب في قاعدة البيانات مباشرةً أبداً. يمرّ كل شيء عبر الخادم ليتم التحقق منه.



مسار الفريق الواحد: من المتصفح إلى القاعدة، ثم إشعار يعود إلى كل الشاشات المفتوحة.

الملفات المهمة

الملف	مسؤوليته
lib/squad.ts	كل قواعد بناء الفريق وحسابات «الجمهور». دوال خالصة بلا اتصالات — يستخدمها المتصفح والخادم معاً.
lib/fpl.ts	التعامل مع واجهة FPL: التقليل والتخينة (الفصل ٨).
lib/rate-limit.ts	كابح الطلبات لكل عنوان IP (الفصل ٩).
components/LiveBoard.tsx	شاشة العرض — وفيها منطق التجميع (الفصول ٤-٦).
api/pools/[id]/entries	الطريق الوحيد للكتابة، ومصدر القراءة المُخَبَّر.

الكارثة المُحتملة — لنحسبها بالأرقام

٣

قبل أن نفهم الحل، يجب أن نشعر بحجم المشكلة. الأمر لا يتعلّق ببطء بسيط، بل بانفجار رياضي.

الطريقة «الساذجة» التي يكتبها الجميع أول مرة

Supabase تُقدّم ميزة رائعة اسمها **Realtime**: كلما تغيّر سطر في قاعدة البيانات، تُرسل إشعاراً فورياً لكل متصفح مشترك. الكود البديهي يكون هكذا:

```
// ❌ الطريقة الساذجة
channel.on("postgres_changes", { table: "entries" }, () => {
  fetch(`/api/pools/${id}/entries`); // جلب كل الفرق من جديد
});
```

يبدو منطقياً: «وصل فريق جديد ← أعد جلب القائمة». المشكلة تظهر فقط عندما يكون هناك جمهور حقيقي.

الانفجار: ضرب في ضرب

الإشعار يذهب إلى **كل** شاشة مفتوحة، وكل شاشة تردّ بجلب **كل** القائمة. إذاً التكلفة = عدد المشاهدين × عدد الفرق **الواصلة**. الرقمان يكبران معاً في اللحظة نفسها — عند الموعد النهائي.

مثال محسوب

لنفترض بثاً متوسطاً: ١٠٠٠ **مشاهد** فتحوا الرابط، و٦٠٠ **فريق** وصلت خلال الدقيقة الأخيرة قبل الإغلاق (أي ١٠ فرق في الثانية).

~6 GB

بيانات منقولة في دقيقة واحدة

600,000

طلب في الدقيقة إجمالاً

10,000

طلب في الثانية على الخادم

الحساب: ١٠ إشعارات/ثانية × ١٠٠٠ شاشة = ١٠,٠٠٠ طلب/ثانية. وكل طلب لا يجلب الفريق الجديد فقط، بل **القائمة كاملة** (٦٠٠ فريق × ١٥ لاعباً). لو كان حجم الرّد ١٠ كيلوبايت فقط، فذلك ١٠٠ ميغابايت في الثانية.

ولماذا هذا خطير أكثر من كونه بطيئاً؟

لأنه **غير مرئي أثناء التطوير**. على جهازك بشاشة واحدة وثلاثة فرق تجريبية، الكود الساذج يعمل بشكل مثالي. المشكلة تظهر أول مرة أمام جمهور حقيقي — في اللحظة التي لا يمكن فيها إصلاح شيء.

كل الفصول التالية هي محاولات لكسر هذه المعادلة: إمّا بتقليل عدد الطلبات الصادرة من كل شاشة، أو بجعل آلاف الطلبات تُكَلَّف قراءة واحدة من قاعدة البيانات.

الحيلة الأولى: تجميع الطلبات (Debouncing)

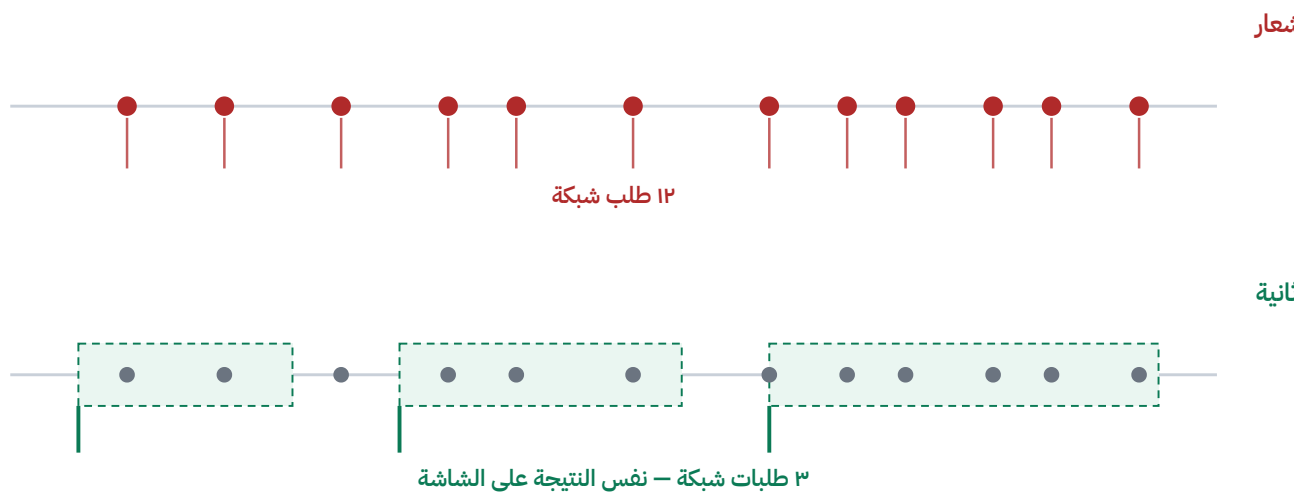
٤

هذه أهم فكرة في المستند كله، وهي ببساطة: «لا تتصرّف فور وصول الحدث، بل انتظر قليلاً – فربما وصل عشرة أحداث أخرى خلال انتظارك، فتتصرّف مرة واحدة عنها جميعاً».

الفكرة بمثال من الحياة

زرّ المصعد. عشرة أشخاص ينتظرون في الطابق الأرضي. كل واحد يضغط الزر عند وصوله. لو كان المصعد «ساذجاً» لصعد ونزل عشر مرات – رحلة لكل ضغطة. لكن المصعد الحقيقي يفعل شيئاً أذكى: يسجّل أن هناك طلباً، ثم **يأتي مرة واحدة** ويأخذ الجميع. الضغوطات العشر أصبحت رحلة واحدة.

هذا بالضبط ما يفعله **Debouncing** (ويُسمّى هنا تحديداً coalescing – «الدمج»): نضع مؤقتاً قصيراً. كل إشعار جديد يصل أثناء عمل المؤقت لا يُطلق طلباً جديداً، بل يُبتلع في نفس الرحلة.



النقاط = إشعارات وصول فرق جديدة. الخطوط السفلية = طلبات فعلية إلى الخادم.

الكود الحقيقي

src/components/LiveBoard.tsx

```
const COALESCE_MS = 1500;

const schedule = () => {
  if (stopped || timer) return; // مؤقت يعمل بالفعل؟ لا تفعل شيئاً ←
  timer = setTimeout(run, COALESCE_MS);
};

channel.on("postgres_changes", { ... }, schedule); // فقط schedule الإشعار يستدعي
```

السطر `if (timer) return;` هو كل السحر. الإشعار الأول يشغل المؤقت. الإشعارات التالية تجد المؤقت يعمل، فتتسحب بصمت. بعد ١,٥ ثانية تنطلق رحلة واحدة تجلب كل ما وصل خلالها.

النتيجة على مثالنا

بدل ١٠ طلبات في الثانية من كل شاشة، أصبح الحد الأقصى طلباً واحداً كل ١,٥ ثانية = **٠,٦٧ طلب/ثانية**. انخفاض بنسبة **٩٣٪**، من ١٠,٠٠٠ إلى ٦٦٧ طلب/ثانية — وقبل أن نضيف أي حيلة أخرى.

لماذا ١,٥ ثانية بالذات؟

هذا الرقم ليس عشوائياً، وهو مثال ممتاز على **مقايضة هندسية**:

- **أقصر مما ينبغي** (٢٠٠ ميلي ثانية مثلاً): لا يبتلع الزحام، وتعود المشكلة.
- **أطول مما ينبغي** (١٠ ثوانٍ): توفير ممتاز، لكن الشاشة تبدو «معطّلة» أمام الجمهور — والمُقدّم يتحدث عن نتيجة لم تظهر بعد.
- **١,٥ ثانية**: طويلة كفاية لابتلاع موجة الموعد النهائي، وقصيرة كفاية بحيث تبقى الشاشة «حيّة» بالنسبة للعين البشرية.

تنبيه على المصطلح

في الاستخدام الشائع، **Debounce** يعني «صّرف المؤقت مع كل حدث جديد» (مثل حقل البحث: لا تبحث حتى يتوقّف المستخدم عن الكتابة). أمّا ما نستخدمه هنا فهو الشكل الأنسب لشاشة حيّة — أقرب إلى **Throttle**: المؤقت لا يُصفر، فالتحديث مضمون خلال ١,٥ ثانية مهما استمر تدفق الإشعارات. لو صغّرناه، لَبقيت الشاشة مجمّدة طوال الزحام لأن الأحداث لا تتوقّف أبداً.

الحيلة الثانية: منع الطلب المكرر

المؤقت يحمينا من كثرة الأحداث. لكن هناك ثغرة ثانية: ماذا لو كان الطلب السابق لا يزال في الطريق عندما يحين وقت الطلب التالي؟

على اتصال بطيء أو خادم مضغوط، قد يستغرق الطلب ٣ ثوانٍ بينما نافذتنا ١,٥ ثانية. فتتراكم الطلبات فوق بعضها، وكل واحد يسأل السؤال نفسه، وتزداد حالة الخادم سوءاً — وهذا ما يُسمّى «الانهيار المتتالي».

المثال: نادلٌ أرسلته لجلب القائمة. بينما هو في المطبخ، تناديه لتطلب القائمة مرة أخرى. لا فائدة — القائمة التي يحملها ستحتوي على الجديد أصلاً. الصواب: انتظر عودته، وإن كان هناك جديد فاطلبه **بعدها**.

src/components/LiveBoard.tsx

```
let inFlight = false; // هل هناك طلب في الطريق؟
let queued = false; // هل وصل جديد أثناء انتظارنا؟

const run = async () => {
  timer = null;
  if (inFlight) { queued = true; return; } // لا تسأل نفس السؤال مرتين ←
  inFlight = true;
  try {
    const res = await fetch(`/api/pools/${pool.id}/entries`);
    if (res.ok && !stopped) setEntries((await res.json()).entries ?? []);
  } catch {
    /* طلب ضائع غير مؤذٍ؛ شبكة الأمان في الفصل التالي ستلحق به */
  } finally {
    inFlight = false;
    lastAt = Date.now();
    if (queued) { queued = false; schedule(); } // الآن نفذ المؤجل ←
  }
};
```

- `inFlight` يضمن وجود طلب واحد فقط على الشبكة في أي لحظة.
- `queued` يضمن أننا لا نفقد تحديثاً وصل أثناء الانتظار.
- كتلة `catch` الفارغة مقصودة: فشل تحديث واحد ليس كارثة، لأن هناك آلية تعويض.

الخلاصة

مهما ساءت الشبكة، كل شاشة تستهلك **اتصالاً واحداً** فقط. هذا يحوّل الضغط من «انهيار متسارع» إلى «بطء مُنتظم» — وهو أرحم بكثير.

الحيلة الثالثة: شبكة الأمان الصامتة

٦

الاعتماد الكامل على Realtime خطر: بعض الشبكات (شبكات الشركات، بعض مزودي الإنترنت، الوكلاء/Proxies) تحجب اتصالات WebSocket. عند هؤلاء المشاهدين، الشاشة تتجمد إلى الأبد دون أي رسالة خطأ.

الحل التقليدي هو **Polling**: اسأل الخادم كل ١٥ ثانية «هل من جديد؟». لكن هذا يعيد إليك مشكلة الفصل ٣: ألف شاشة تسأل باستمرار حتى لو لم يحدث شيء. لذلك جعلنا الاستطلاع **مشروطاً**:

src/components/LiveBoard.tsx

```
const POLL_MS = 15000;

const poll = setInterval(() => {
  if (Date.now() - lastAt ≥ POLL_MS) schedule(); // Realtime فقط إذا صمت
}, POLL_MS);
```

الفكرة في جملة

نحتفظ بوقت آخر تحديث ناجح (`lastAt`). إذا مرّت ١٥ ثانية بلا أي تحديث، نفترض أن Realtime لا يعمل ونسأل يدوياً. الشاشة التي تستقبل إشعارات بشكل طبيعي لا تُنفق ولا طلباً واحداً على شبكة الأمان.

وهكذا تحصل على أفضل ما في العالمين: سرعة Realtime للأغلبية، وضمان بحد أقصى ١٥ ثانية تأخير للأقلية المحجوبة. ولاحظ أنه يعالج أيضاً الطلب الضائع من الفصل ٥.

تنظيف الموارد — تفصيلة صغيرة تمنع تسريب الذاكرة

```
return () => {
  stopped = true;
  if (timer) clearTimeout(timer);
  clearInterval(poll);
  if (channel) void channel.unsubscribe();
};
```

عندما يغلق المشاهد الصفحة، تُلغى كل المؤقتات والاشتراكات. بدون هذا، تبقى مؤقتات «شبح» تُطلق طلبات لصفحة لم تعد موجودة — وهو من أشهر أسباب بطء التطبيقات بعد فترة استخدام طويلة.

الحيلة الرابعة: ذاكرة مؤقتة عند الحافة

كل ما سبق يقلّل الطلبات الصادرة من كل شاشة. لكن يبقى شيء لا يستطيع المتصفح إصلاحه بنفسه: ألف شاشة تسأل السؤال نفسه في اللحظة نفسها.

لاحظ أن الجواب واحد للجميع — لا يوجد محتوى مخصّص لكل مشاهد. فلماذا نقرأ من قاعدة البيانات ألف مرة لنعطي الجواب نفسه؟ يكفي أن نقرأ مرة واحدة ونحتفظ بالجواب لثانيتين.

src/app/api/pools/[id]/entries/route.ts

```
return NextResponse.json(
  { entries: data ?? [] },
  { headers: {
    "Cache-Control": "public, max-age=0, s-maxage=2, stale-while-revalidate=10"
  } },
);
```

تفكيك السطر كلمةً كلمة

التعليمة	معناها
public	الجواب ليس سرّياً وليس خاصاً بشخص، فيجوز لأي وسيط تخزينه.
max-age=0	متصفح المشاهد لا يخزّنه — وإلاّ لرأى فريقه القديم على قرصه الصلب.
s-maxage=2	خادم الحافة (CDN) يحتفظ بالجواب ثانيتين. هنا يقع التوفير الحقيقي.
stale-while-revalidate=10	بعد انتهاء الثانيتين، قدّم النسخة القديمة فوراً ثم حدّثها في الخلفية — فلا ينتظر أحد.

1

قراءة — مع s-maxage=2

1000

قراءة من قاعدة البيانات — بدون تخبئة

هذه هي النقلة النوعية

بعد هذا السطر، أصبح الحمل على قاعدة البيانات مستقلاً تماماً عن عدد المشاهدين. سواء كان الجمهور ١٠٠ أو ١٠٠,٠٠٠، فقاعدة البيانات تُقرأ بحد أقصى ٣٠ مرة في الدقيقة لكل حوض. هذه هي الحدود التي تنتقل بالتطبيق من «يعمل على جهازٍ» إلى «يعمل أمام جمهور».

والثانيتان مختارتان بعناية أيضاً: أقصر من نافذة التجميع (١,٥ ثانية) بقليل، فلا تتراكم مصادر التأخير على بعضها؛ وأطول من صفر، فتلتقط موجة الطلبات المتزامنة.

الحيلة الخامسة: ترويض واجهة FPL

٨

أسعار اللاعبين ونقاطهم تأتي من خادم FPL الرسمي. حجم الردّ الواحد: ٢,٣ ميغابايت. وهذه البيانات يحتاجها كل مشاهد ليختار فريقه.

البديهي هو تخبئة الطلب نفسه ليُجلب مرة كل ساعة. وهنا وقعنا على فخّ حقيقي:

الفخّ الصامت

حدّ التخبئة في Next.js هو ٢ ميغابايت. ملف FPL يبلغ ٢,٣ ميغابايت، أي أنه يتجاوز الحد. والنتيجة أن التخبئة **تفشل بصمت**: لا خطأ، لا تحذير، لا شيء في السجل. كل ما يحدث أن كل زائر يسحب ٢,٣ ميغابايت من خادم خارجي — وأنت تظن أن لديك ذاكرة مؤقتة تعمل.

الحل: قلّص أولاً، ثم خبّئ

التطبيق لا يحتاج ٩٠٪ من ذلك الملف (تاريخ المباريات، الإحصاءات التفصيلية، صور اللاعبين...). فنجلبه **بلا تخبئة**، نقتطع منه ما نحتاجه فقط (~١٠٠ كيلوبايت)، ثم نخبّي النتيجة **المقلّصة**.

src/lib/fpl.ts

```
async function fetchBootstrap() {
  const res = await fetch(BOOTSTRAP_URL, { cache: "no-store" }); // ← لا تخبئة هنا
  const raw = await res.json(); // ٢,٣ ميغابايت
  // ... نأخذ الحقول الضرورية فقط ...
  return { gw, teams, players, events, ... }; // ١٠٠ كيلوبايت~
}

export const getBootstrap = unstable_cache(fetchBootstrap, ["fpl-bootstrap"], {
  revalidate: 3600, // التخبئة هنا: اتصال خارجي واحد كل ساعة، للجميع ←
  tags: ["fpl"],
});
```

ساعة/1

اتصال بخادم FPL بدل واحد لكل زائر

96%

تقليص في الحجم

لماذا الحقول بأسماء غريبة مثل n و pts ؟

لأن هذه القائمة تُرسل إلى **كل** مشاهد. الفرق بين "name" و "n" ثلاثة أحرف — لكن مضروبة في ٧٠٠ لاعب × عدد المشاهدين، تصبح ميغابايتات. اختصار الأسماء هنا ليس كسلاً بل قرار مقصود.

مدد مختلفة لبيانات مختلفة

البيانات	المدة	السبب
قائمة اللاعبين والأسعار	3600s	الأسعار تتغير مرة يومياً على الأكثر.
النقاط الحية أثناء المباريات	120s	تتغير مع كل هدف — يجب أن تبقى طازجة.

التدهور بلطف

لو تعطلت واجهة FPL أثناء إرسال أحد المشاهدين لفريقه، لا ينهار التطبيق بصفحة خطأ. بل يرّد برسالة مفهومة وبكود 503 : «واجهة FPL لا تستجيب، لذا تعذّر التحقق من الفريق. حاول بعد قليل». كل رسالة خطأ في المشروع جملة كاملة تخبر المستخدم بما يفعله تالياً.

الحيلة السادسة: كايح المُسيئين

٩

كل ما سبق يحمي من الزحام الطبيعي. يبقى الزحام **المفتعل**: شخص يكتب سكريبتاً بسيطاً يرسل آلاف الفرق في الثانية ليفسد نتيجة البث.

src/lib/rate-limit.ts

```
const LIMIT = 30; // ٣٠ رسالة
const WINDOW = 10 * 60 * 1000; // خلال ١٠ دقائق

const limit = rateLimit(`entries:${clientId(req)}`, LIMIT, WINDOW);
if (!limit.ok) {
  return NextResponse.json(
    { error: "Too many submissions from this connection. Wait a minute..." },
    { status: 429, headers: { "Retry-After": String(limit.retryAfter) } },
  );
}
```

الآلية تُسمّى **النافذة المنزلقة** (Sliding Window): نحتفظ بأوقات آخر الطلبات من كل عنوان IP، ونتجاهل ما مضى عليه أكثر من عشر دقائق. تجاوزت الثلاثين؟ تُرفض مؤقتاً مع إخبارك بعدد الثواني المتبقية.

لماذا ٣٠ وليس ٣؟

لأن المشاهد الطبيعي **يعدّل فريقه** عدة مرات قبل الموعد النهائي — يجرب لاعباً، يغيّر رأيه، يبذل الكابتن. الحد يجب أن يكون سخياً بما يكفي ألا يزعج إنساناً عادياً، وضيّقاً بما يكفي ليوّقف سكريبتاً. هذه المقايضة هي جوهر تحديد المعدّل: **لا تعاقب المستخدم الحقيقي لتمنع المسيء.**

اعتراف بحدود الحلّ

العدّادات محفوظة في ذاكرة الخادم لا في قاعدة البيانات. هذا يعني: لا اعتماديات إضافية، ولا رحلة شبكة لكل طلب — لكن على استضافة serverless، كل نسخة من الخادم تحتفظ بـعدّادها الخاص. فهذا **كايح ضد العبث الآلي** وليس حصّة دقيقة. والملف نفسه يوثّق ذلك بوضوح، حتى يعرف من يقرؤه لاحقاً متى يحتاج إلى نقله إلى قاعدة البيانات.

درس عام

الحل «الكافي» الموثّق حدوده أفضل من حلّ مثالي معقّد لا يحتاجه المشروع بعد. المهم ألا تنسى أنك اخترت ذلك — ولهذا التعليق مكتوب في أعلى الملف.

تفصيلة صغيرة: تنظيف الذاكرة

```
if (hits.size > 5000) sweep(now, windowMs); // امسح العناوين القديمة
```

بدون هذا السطر، ستتمو خريطة العناوين إلى ما لا نهاية على خادم يعمل لأسابيع، حتى تستهلك الذاكرة كلها. حماية من الزحام يجب ألا تصبح هي نفسها سبب التعطّل.

الحيلة السابعة: واجهة خفيفة في المتصفح

١٠

ليس كل بطء سببه الخادم. شاشة العرض تعمل على جهاز المُقدِّم أثناء البثّ المباشر — وإن التهمت المعالج، تُعثر البثّ نفسه.

١) لا تُعد الحساب إن لم تتغيّر المُدخلات

عند كل تحديث، تُعاد حسابات ثقيلة: إحصاء الأصوات، ترتيب اللاعبين، بناء التشكيلة. `useMemo` تعني حرفياً: «احفظ النتيجة، ولا تُعد الحساب إلا إذا تغيّر ما تعتمد عليه».

```
const byId = useMemo(() => new Map(boot.players.map(p => [p.id, p])), [boot.players]);
const t     = useMemo(() => tally(entries), [entries]);
const cx    = useMemo(() => crowdXI(t, byId, pool.formation), [t, byId, pool.formation]);
```

خريطة الـ ٧٠٠ لاعب تُبنى مرة واحدة عند فتح الصفحة، لا مع كل فريق يصل. والفرق بين البحث في مصفوفة والبحث في خريطة (Map) هو الفرق بين تصفّح كتاب صفحةً صفحةً وفتحه على الفهرس مباشرةً.

٢) لا ترسم ما لا يمكن رؤيته

src/components/TeamPicker.tsx

```
const LIST_CAP = 200; // من أصل ٧٠٠ لاعب
return { rows: hits.slice(0, LIST_CAP), total: hits.length };
```

عرض ٧٠٠ لاعب دفعةً واحدة يعني ٧٠٠ عنصر في الصفحة، ولن يمرّر أحد إلى نهايتها. نعرض أقوى ٢٠٠ نتيجة ونخبر المستخدم بالعدد الكلي — فالقائمة لا تنتهي فجأة عند حدّ خفيّ.

٣) الحفظ المحلي — راحة للمستخدم وتوفير للخادم

```
localStorage.setItem(`cxi.squad.${pool.id}`, JSON.stringify(squad));
```

الفريق قيد البناء يُحفظ في المتصفح. إن أغلقت الصفحة بالخطأ، يعود المستخدم ويجد اختياراته مكانها — بلا أي طلب إضافي للخادم.

المبدأ الجامع

كل عمل تؤجّله أو تتجنّبه هو عمل لا يمكن أن يفشل. أسرع طلب شبكة هو الطلب الذي لم تُرسله، وأسرع حساب هو الذي لم تُعده.

الأمان: لا تثق بالمتصفح أبداً

الأداء وحده لا يكفي. تطبيق سريع يمكن لأي شخص التلاعب بنتائجه لا قيمة له على الهواء مباشرةً.

المشكلة: كيف نعرف مَنْ أنتَ بلا تسجيل دخول؟

الحل الساذج: يرسل المتصفح مُعرِّفه ضمن البيانات. والمشكلة أن أي شخص يستطيع تغيير ذلك المُعرِّف والكتابة فوق فريق شخص آخر.

src/lib/identity.ts

الحل: مُعرِّف عشوائي داخل **كوكي موقع رقمياً** (HMAC-SHA256) بمفتاح لا يعرفه إلا الخادم، ومُعَلَّم بـ `httpOnly` فلا تستطيع أي شيفرة في الصفحة قراءته. يمكنك أن تعيد كوكيك، لكنك لا تستطيع أن تصنع كوكياً لشخص آخر: التوقيع لن يكون صحيحاً.

```
// الهوية من الكوكي، لا من محتوى الطلب
const voter = await readVoter(jar.get(VOTER_COOKIE)?.value);
```

التحقق يحدث مرتين – وهذا مقصود

أين	الغرض	هل يمكن تجاوزه؟
<code>validateSquad</code> في المتصفح	ردّ فعل فوري أثناء الاختيار: «تجاوزت الميزانية»، «٣ لاعبين كحد أقصى من النادي».	نعم – وهذا مقبول، فهو للراحة فقط.
<code>validateEntry</code> في الخادم	إعادة التحقق من الصفر مقابل الأسعار الحيّة. هذا هو القرار الذي يُحتسب.	لا – لا يوجد طريق آخر للكتابة.

قد يبدو هذا تكراراً، لكنه ليس كذلك: الأول يخدم **تجربة المستخدم**، والثاني يخدم **صحة البيانات**. حذف الأول يجعل التطبيق مزعجاً؛ حذف الثاني يجعله بلا قيمة.

وأخيراً: مفتاحان لا مفتاح واحد

- **المفتاح العام** (anon): يستخدمه المتصفح، ولا يرى سوى ثلاثة حقول (`id` , `pool_id` , `updated_at`) – ما يكفي للإشعارات اللحظية فقط. هوية المصوّتين غير مقروءة به.
- **مفتاح الخدمة** (service role): يبقى على الخادم حصراً، ولا يُستورد داخل أي مكّون يعمل في المتصفح.

قاعدة إضافية

الإرسال يُرَفَض تلقائياً بعد انقضاء الموعد النهائي للجولة — بالوقت المحسوب على **الخدام** لا على ساعة المتصفح، التي يستطيع أي أحد تغييرها.

كل الأرقام في جدول واحد

الرقم	أين	ما الذي يشترطه
1500 ms	تجميع إشعارات الشاشة	يتتبع موجة الموعد النهائي في طلب واحد.
inFlight	قفل الطلب الواحد	يمنع الانهيار المتتالي على الشبكات البطيئة.
15 s	استطلاع مشروط	ينقذ من تُحجب عنهم WebSocket، بلا تكلفة على البقية.
s-maxage=2	تخبيئة عند الحافة	يفصل حمل قاعدة البيانات عن عدد المشاهدين.
3600 s	تخبيئة بيانات FPL	اتصال خارجي واحد كل ساعة بدل واحد لكل زائر.
2.3MB → 100KB	تقليص الحمولة	يجعل التخبيئة ممكنة أصلاً، ويخفّف كل صفحة.
30 / 10min	تحديد المعدّل	يوقف السكربتات دون إزعاج المشاهد الحقيقي.
200	سقف قائمة اللاعبين	يبقي المتصفح خفيفاً أثناء البثّ.

المقارنة النهائية — ١٠٠٠ مشاهد و٦٠٠ فريق في الدقيقة الأخيرة

المقياس	بدون تحسين	بعد التحسينات
طلبات على الخادم / ثانية	~10,000	~667
قراءات قاعدة البيانات / دقيقة	~600,000	~30
اتصالات بواجهة FPL / ساعة	واحد لكل زائر	1
تأخّر ظهور فريق على الشاشة	فوري (نظرياً)	٣,٥ ثانية ≤

المقايضة التي قبلناها بوعي

دفعنا ثلاث ثوانٍ ونصف من التأخير في أسوأ الحالات، واشترينا بها انخفاضاً بمقدار أربع مراتب عشرية في الحمل. ولا أحد في الجمهور يلاحظ ثلاث ثوانٍ على شاشة بـت — بينما الجميع يلاحظ شاشة انهارت.

ثلاثة دروس تصلح لأي مشروع

1. **ابحث عن الضرب المزدوج.** الكارثة لا تأتي من عملية بطيئة، بل من عملية تتضاعف مع مُتغيّرين معاً (مشاهدون × أحداث).
ابحث عن هذا النمط قبل أي شيء آخر.
2. **الأحداث المتكررة تُجمّع، لا تُنفَّذ فرداً فرداً.** نافذة قصيرة من التأخير المتعمّد هي أرخص تحسين في البرمجة — بضعة أسطر تحذف ٩٠٪ من العمل.
3. **اكتب سبب الرقم بجانب الرقم.** 1500 بلا تعليق يبدو عشوائياً، وسيحذفه أحدهم بعد ستة أشهر ظناً منه أنه يُسرّع الشاشة — ثم ينهار كل شيء في أول بـت حقيقي. كل ثابت في هذا المشروع فوقه تعليق يشرح لماذا.

وأخيراً

لا شيء في هذا المستند يتطلب مكتبة خاصة أو بنية تحتية باهظة. كل ما ورد هو **مؤقتات**، **وأعلام منطقية (booleans)**، **وترويسات HTTP** — أدوات موجودة في كل متصفح وكل خادم منذ سنوات. الفرق ليس في الأدوات، بل في أن تسأل «ماذا يحدث لو دخل ألف شخص معاً؟» **قبل** أن يدخلوا فعلاً.